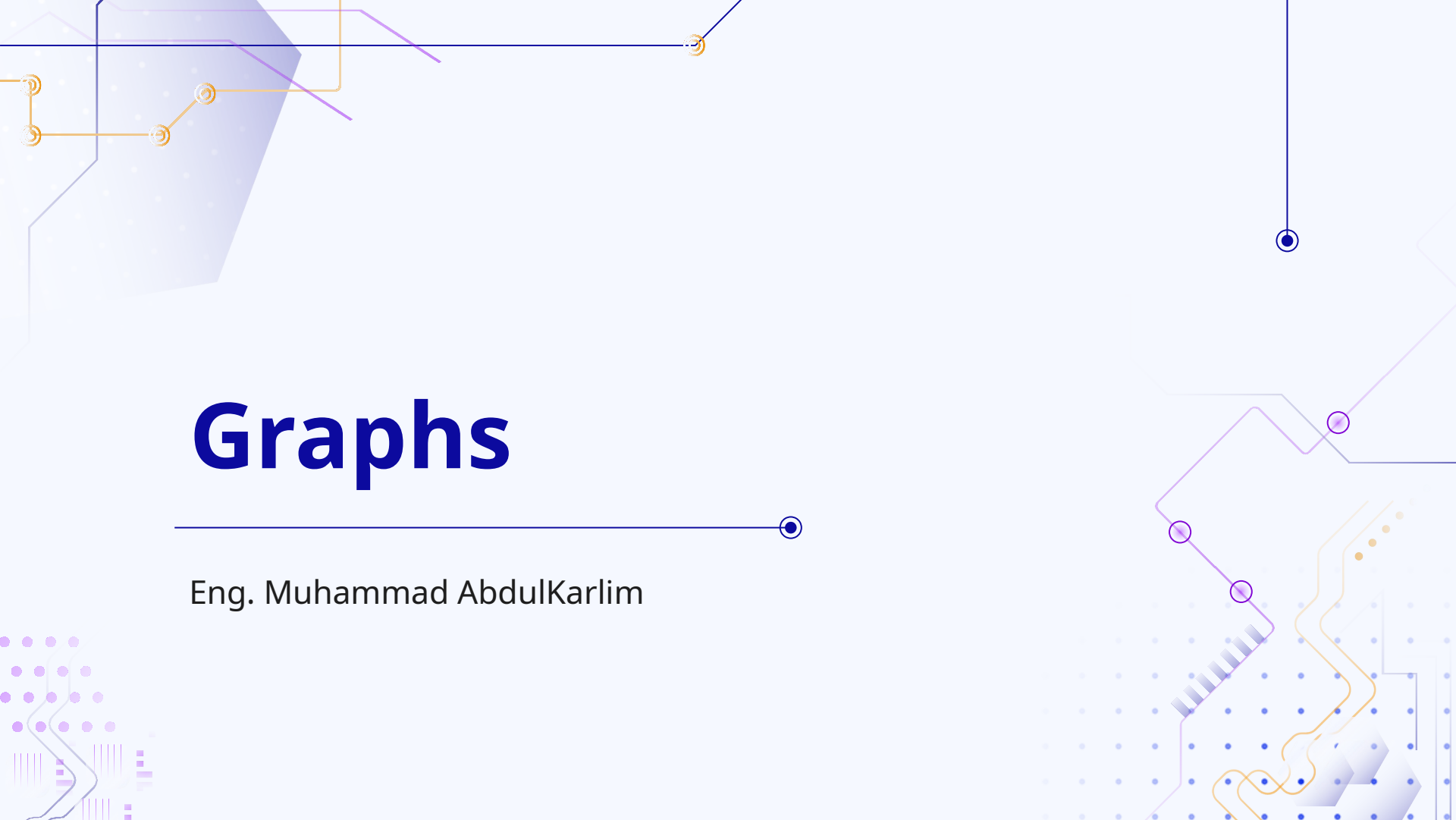




Graphs

Eng. Muhammad AbdulKarlim



Table of contents

01

Introduction to Graphs

What is a Graph?
Why Study Graphs?

03

Graph Representation

Adjacency List
Adjacency Matrix

02

Graph Terminology

Neighbors, Degree, Path,
Cycle, Connectivity

04

Graph Traversal Algorithms

Depth-First Search (DFS)
Breadth-First Search (BFS)



Table of contents

05

Advanced Graph Concepts

Topological Sort
Strongly Connected
Components (SCC)

07

All Pairs Shortest Path

Slow All Pairs Shortest Path
Fast All Pairs Shortest Path

06

Single Shortest Path Algos

Dijkstra's Algorithm
Bellman-Ford Algorithm
Johnson's Algorithm

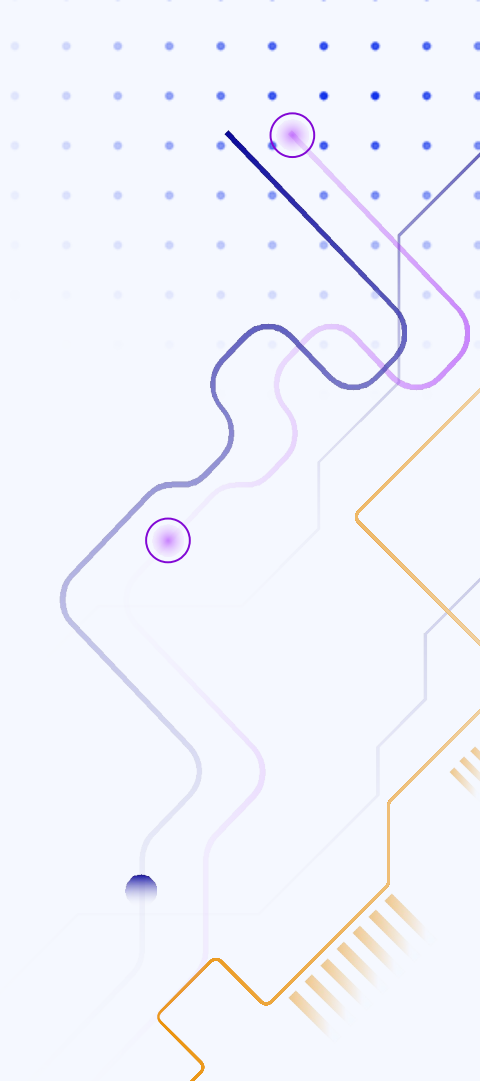
08

Flow Networks

Flow Networks
Minimum Cut



01 Introduction to Graphs



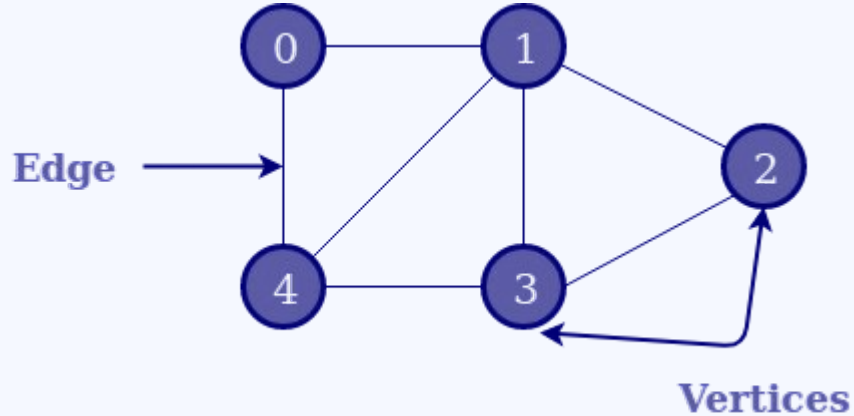
What is a Graph?

Definition

A graph is a network that helps define and visualize **relationships** between various **components**.

Components:

- Nodes (Vertices)
- Edges



Why Study Graphs?

Applications:

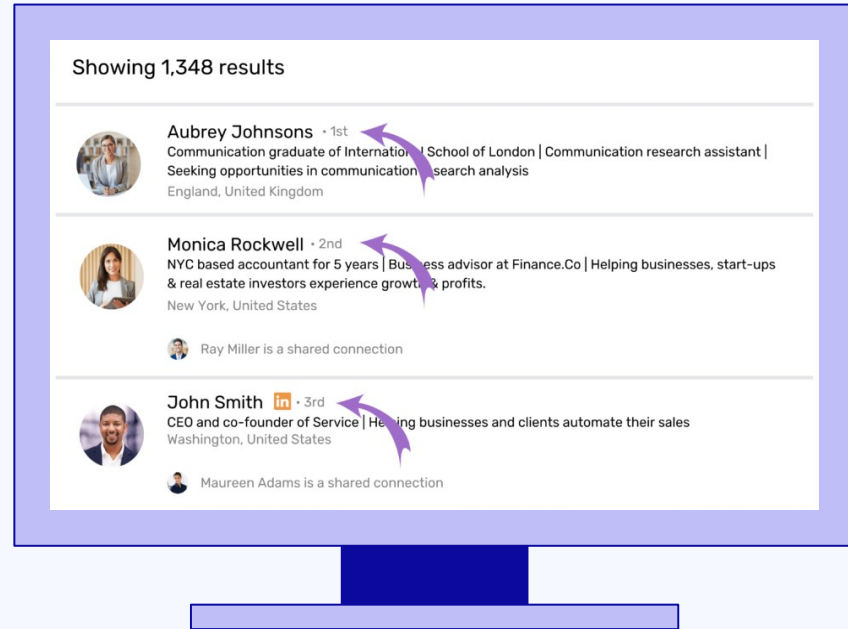
- Graphs show up everywhere, either directly or indirectly.
- Graph theory can solve modern problems.

Examples:

- Social networks (LinkedIn).
- Shortest path problems (e.g., navigation).

Examples:

Social networks (LinkedIn)



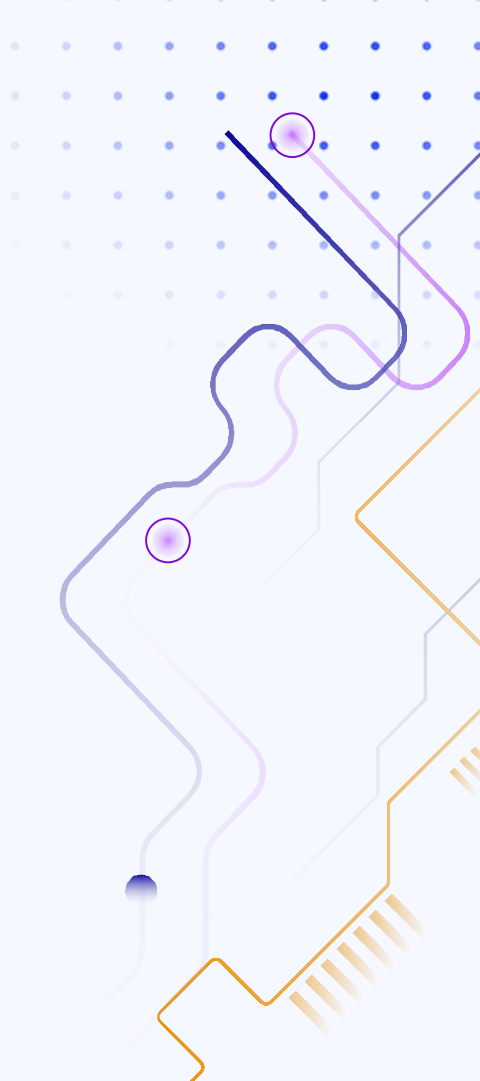
Examples:

Social networks (LinkedIn)

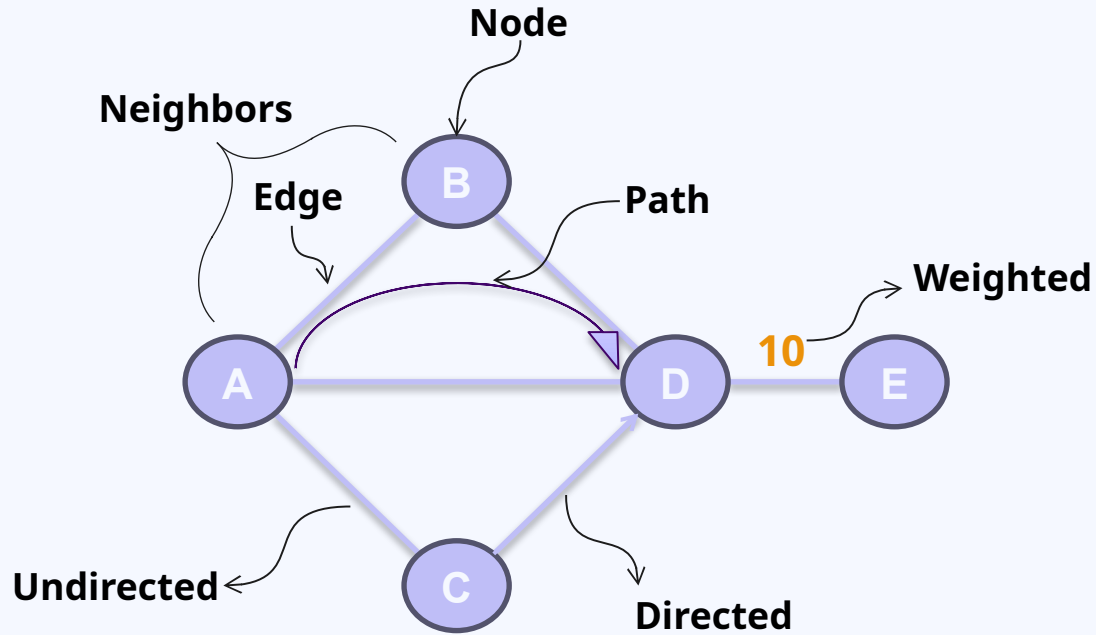




02 Graph Terminology



Graph Terminology



Graph Terminology



Neighbors

Nodes connected by an edge



Path

Sequence of vertices connected by edges



Connectivity

Nodes are connected if there is a path between them



Degree

Number of neighbors a node has



Cycle

Path that starts and ends at the same node



Connected Components

Subsets of nodes that are connected

Graphs Types

Graphs

**Undirected
Graph**

**Directed
Graph**

Cyclic

Acyclic

Graphs

**Unweighted
Graph**

**Weighted
Graph**

03 Graph Representation



Graph Representation Techniques



Adjacency List

An adjacency list represents a graph as a **collection of lists**. Each list corresponds to a node and contains the nodes it is connected to.



Adjacency Matrix

An adjacency matrix uses a **2D array** to represent a graph, with rows and columns corresponding to nodes.

Graph Representation Techniques



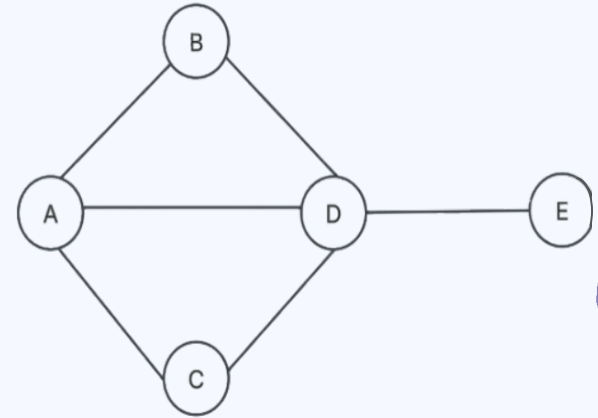
Adjacency List

A → B, C, D
B → A, D
C → A, D
D → A, B, C, E
E → D

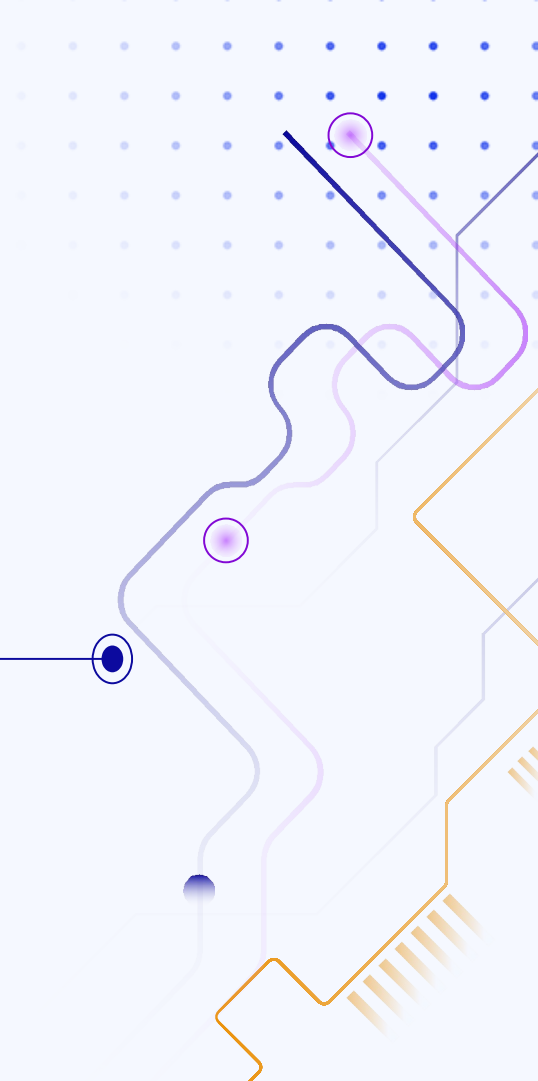


Adjacency Matrix

	A	B	C	D	E
A	0	1	1	1	0
B	1	0	0	1	0
C	1	0	0	1	0
D	1	1	1	0	1
E	0	0	0	1	0

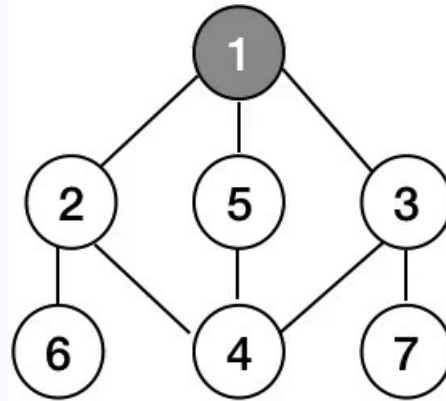


04 Graph Traversal Algorithms



Depth-First Search (DFS)

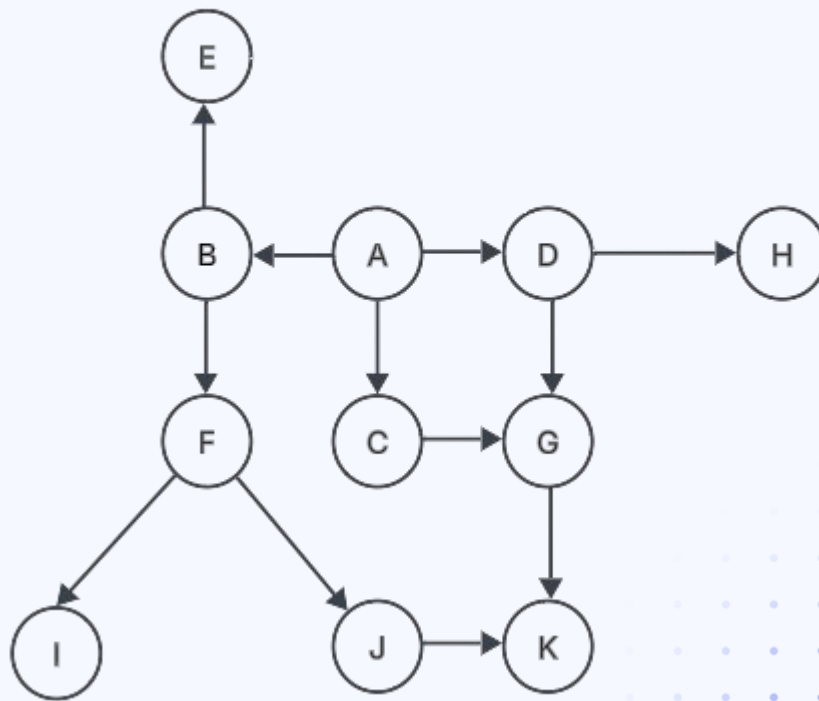
DFS starts from a specific node, processing all children randomly, and ensures that each child is fully processed before moving to another. The traversal order can vary based on the structure of the graph, showcasing how deeply it explores each branch.



DFS Example

Traversal order:

A B F I J K E C G D H



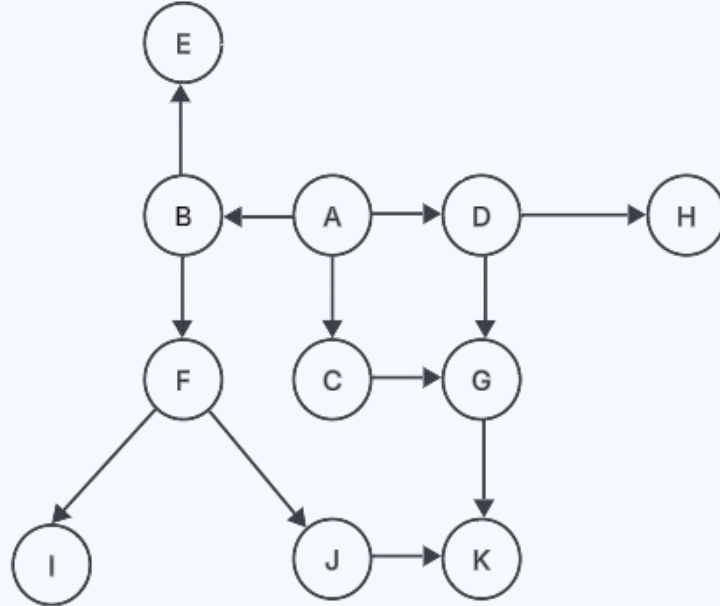
Topological Sort Overview

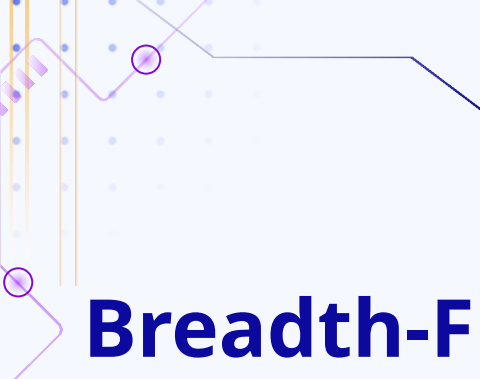
- Create a sequence of nodes where each node comes before its children.
- Perform DFS, and when a node is done, place it at the back (as if you push in a stack).

Topological Sort Overview

Sorted order:

A D H C G B E F J K I





Breadth-First Search (BFS)

BFS traverses all the children of a node before moving on to their children, using a queue data structure.

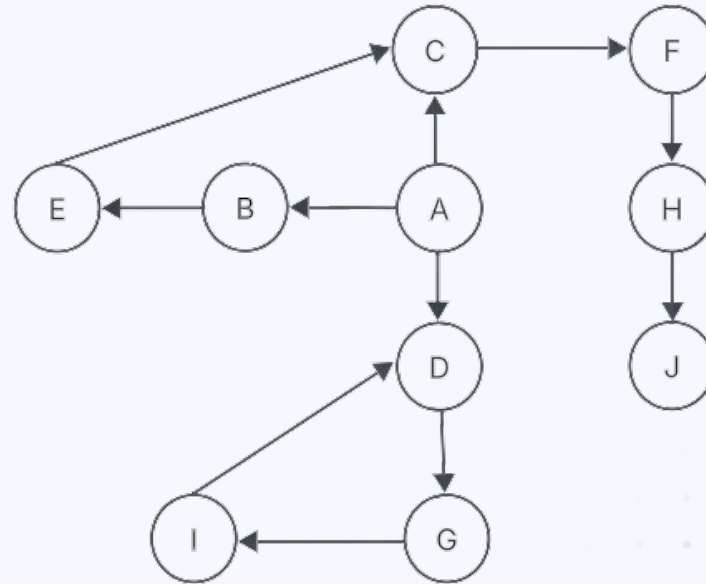
This method is particularly useful for finding the shortest path in an unweighted graph, allowing for systematic exploration.



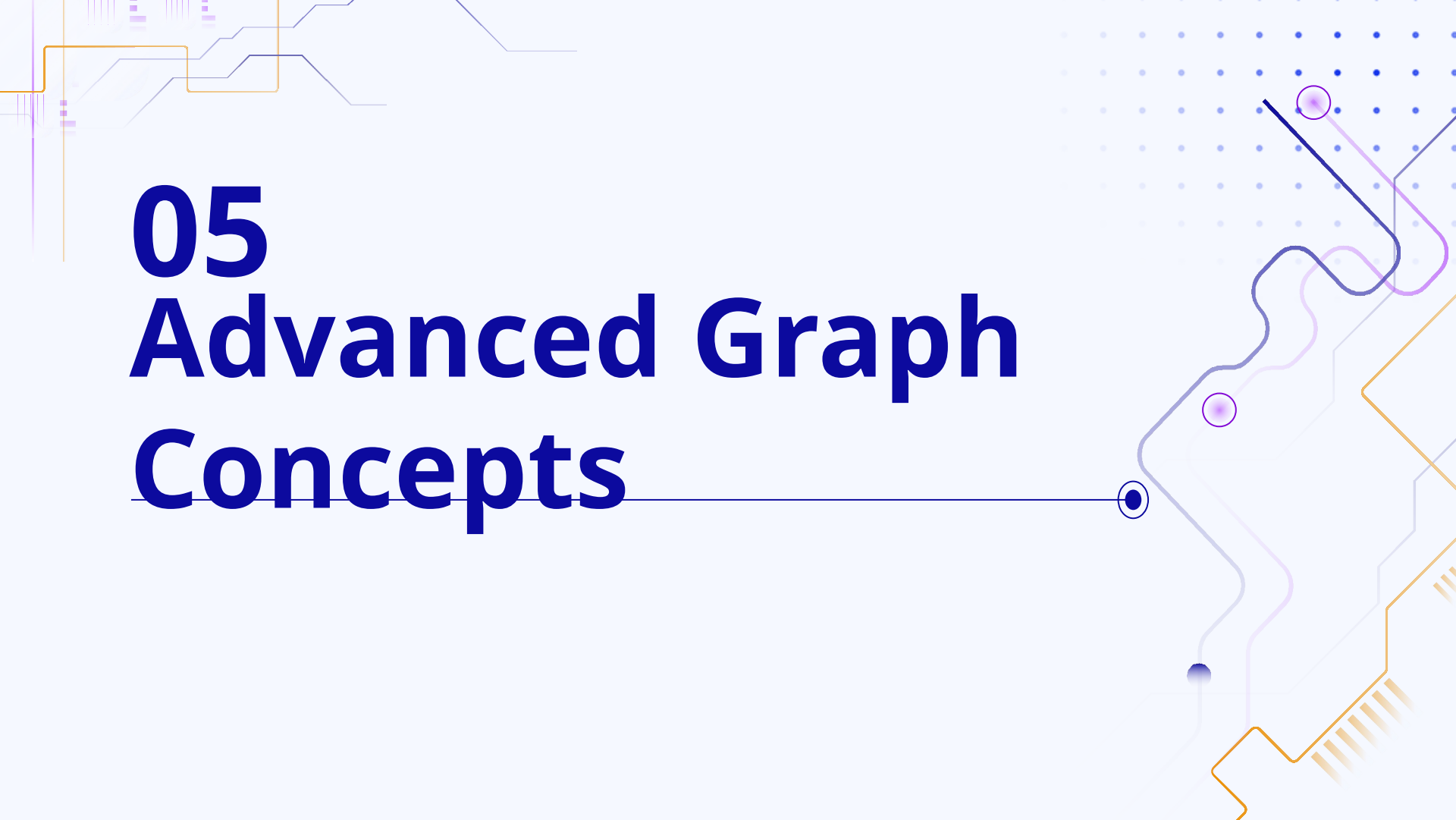
BFS Example

Traversal order:

A B C D E F G H I J



05 Advanced Graph Concepts

The background features a light blue gradient. On the left, there are stylized circuit traces in orange and purple. On the right, a grid of small blue dots is visible, with several larger purple circles and lines connecting them, suggesting a graph structure. A horizontal blue line extends from the text area towards the right side of the slide.

Connectivity Concepts

- Two nodes are connected if there is a path between them.
- A graph is connected when all nodes are interconnected.
- Connected components are subsets of nodes that are connected.

Strongly Connected Component (SCC)

- A subgraph where every node is reachable from every other node.
- It is important for analyzing networks and flow structures.
- Used in applications like social networks and circuit design.

Types of Edges in DFS

01

Tree edge

An edge present in the DFS tree

02

Forward edge

Connects a node to a descendant

03

Backward edge

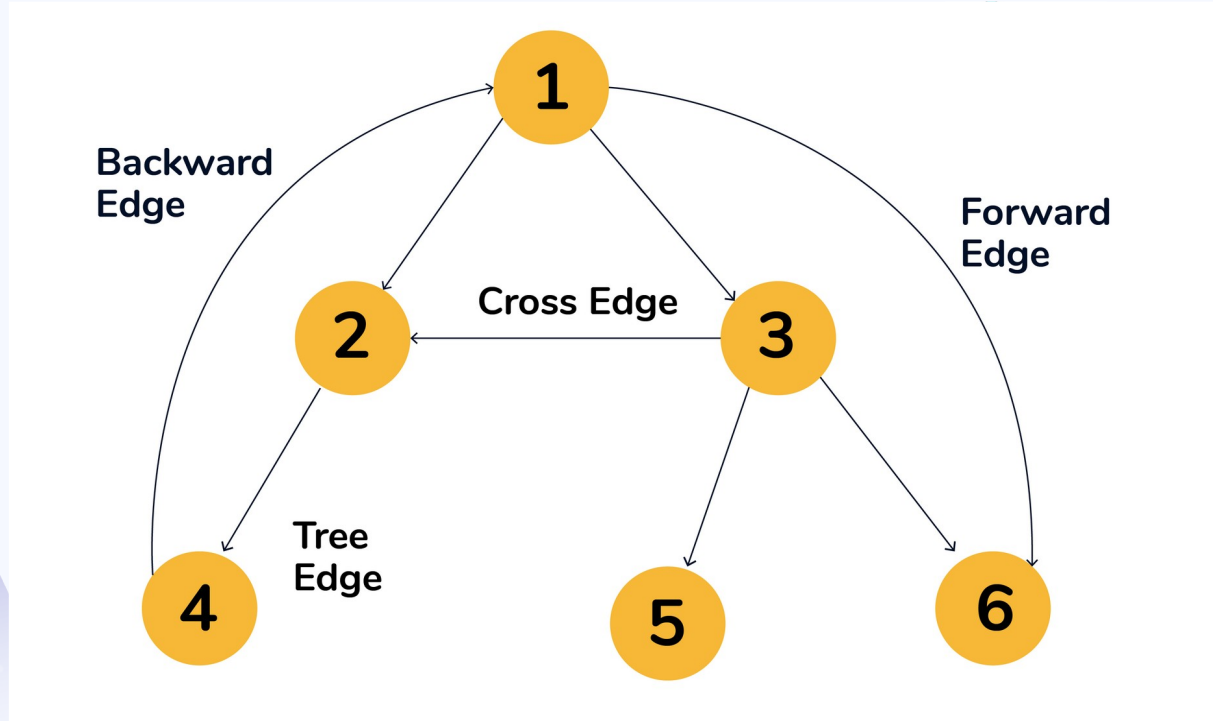
Connects to an ancestor, forming a cycle

04

Cross edge

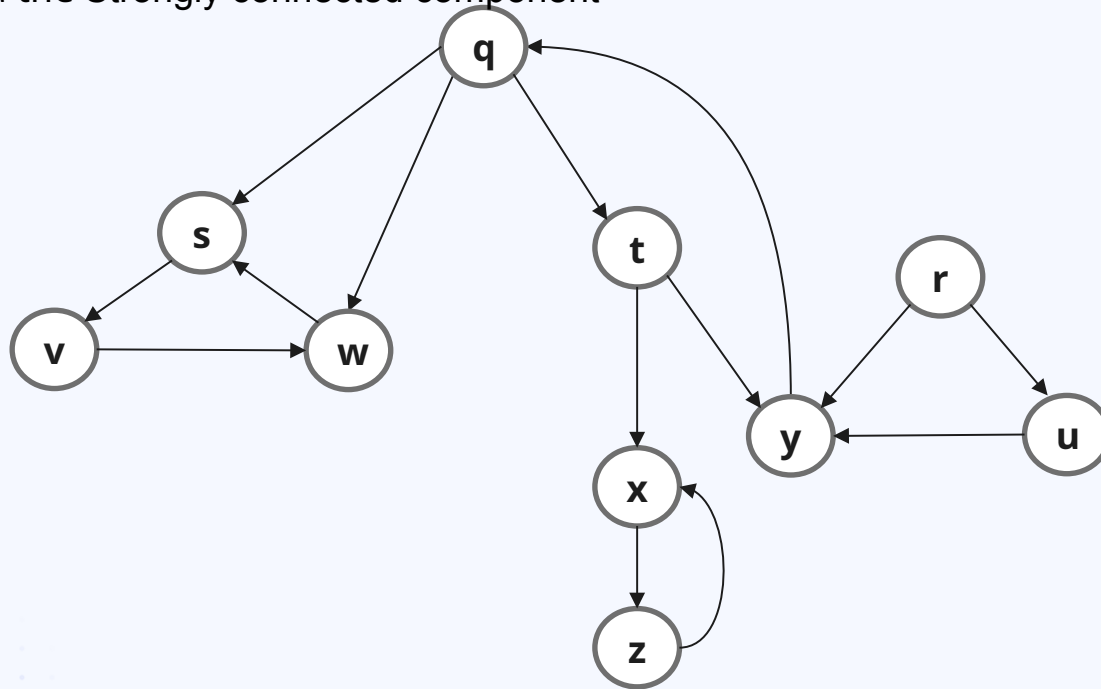
Connects nodes in the DFS tree without ancestor/descendant relationship

Types of Edges in DFS

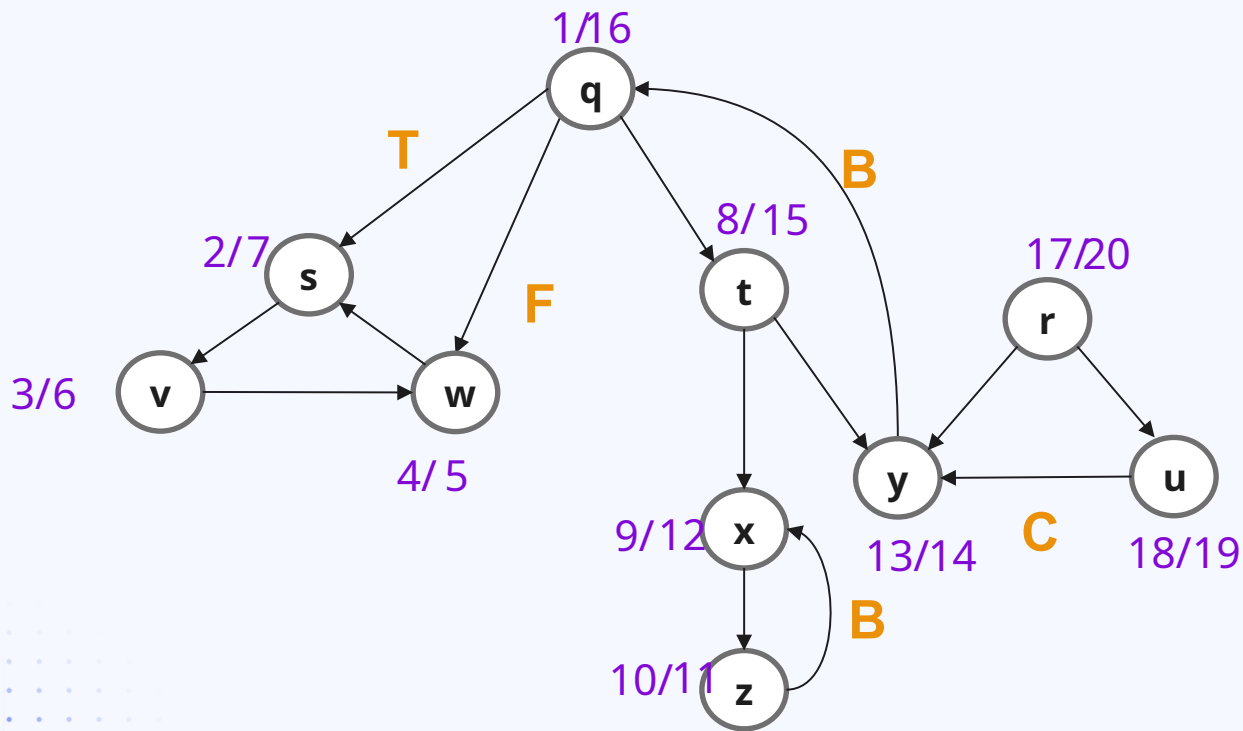


Example

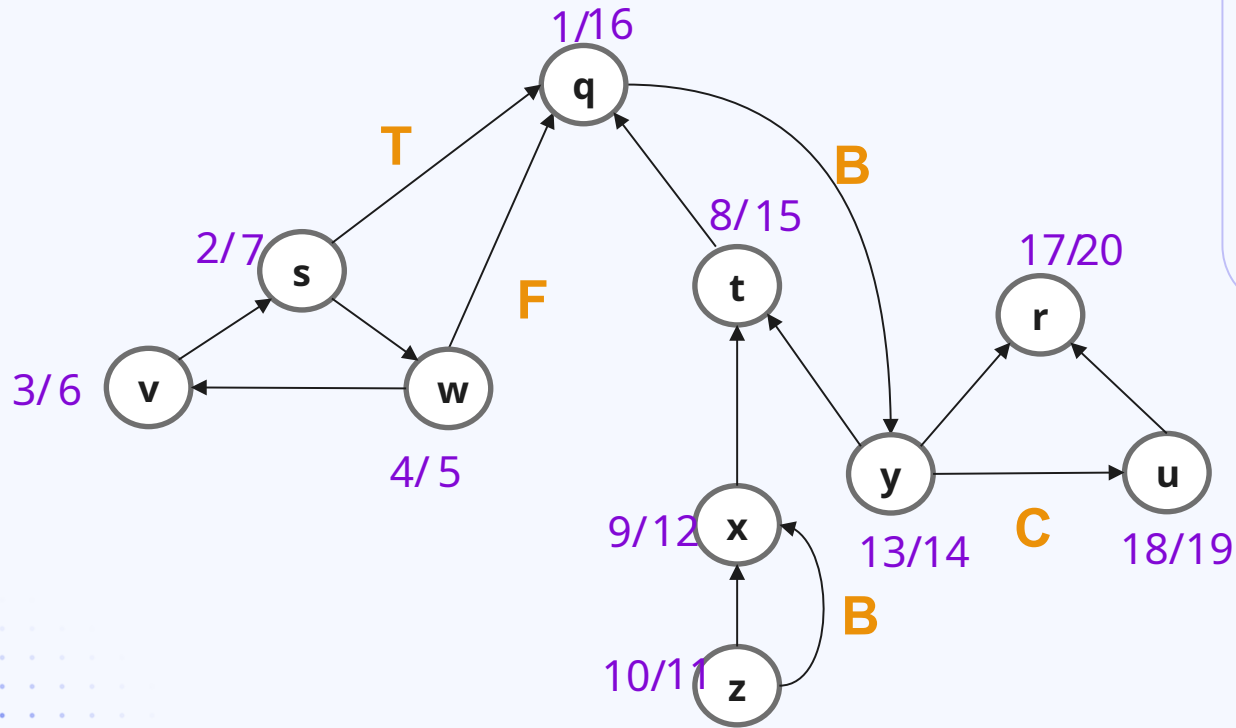
Find the Strongly connected component



Example



Strongly connected component



- { r }
- { u }
- { q, y, t }
- { x, z }
- { s, w, v }

06 Shortest Path Algorithms

A decorative background featuring a light blue circuit board pattern with various colored lines (blue, orange, purple) and dots. The pattern is more dense on the right side of the image.

Single Source Shortest Path algorithms:

01 — **BFS** — for **unweighted** graph

02 — **Dijkstra** — for **positive** weighted graph

03 — **Bellman-Ford** — for **positive** and **negative** weighted graph

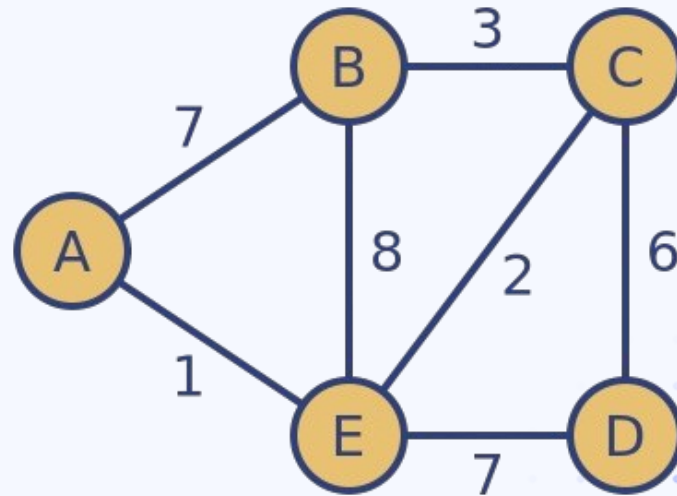
04 — **Directed Acyclic graph algorithm**



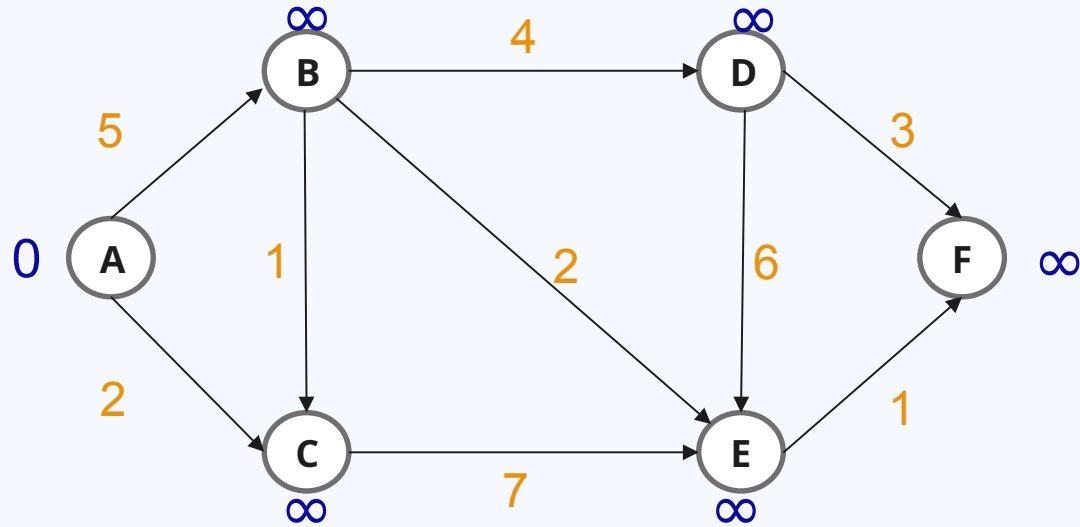
Dijkstra's Algorithm

Dijkstra's algorithm **finds the shortest path from a specific node** to all other nodes.

It updates distances to neighboring nodes as it traverses the graph, ensuring that the shortest possible path is always chosen.

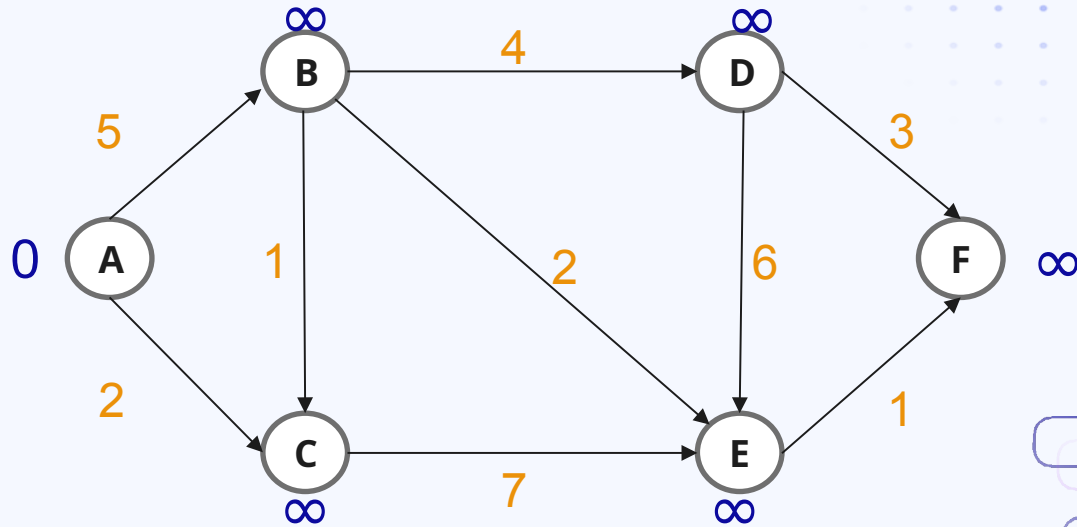


Dijkstra's Example



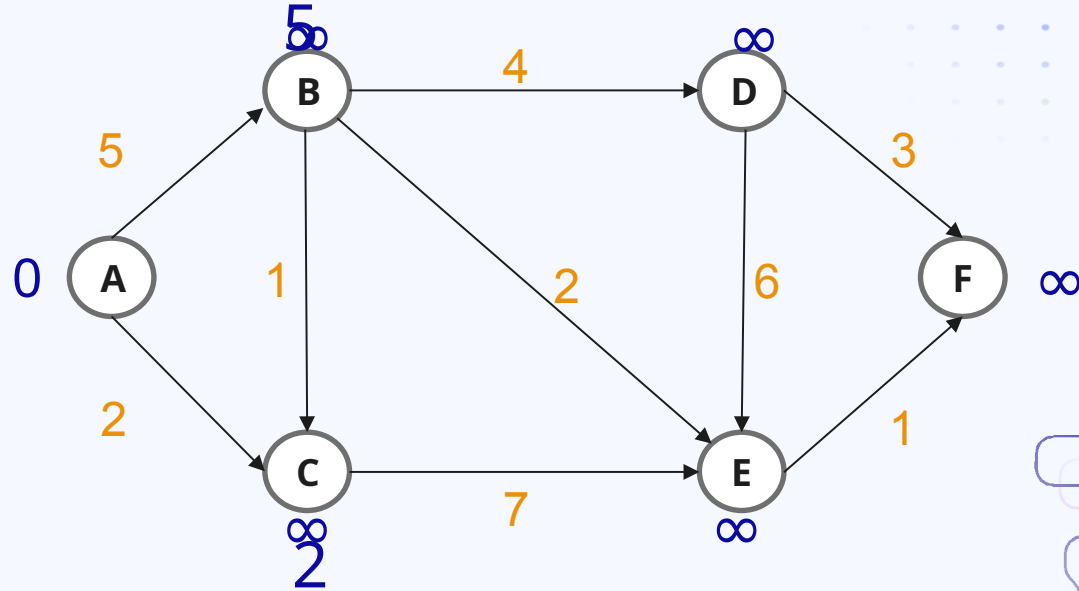
Dijkstra's Example

Nod e	Distance	Parent
A	0	-
B	∞	A
C	∞	A
D	∞	
E	∞	
F	∞	



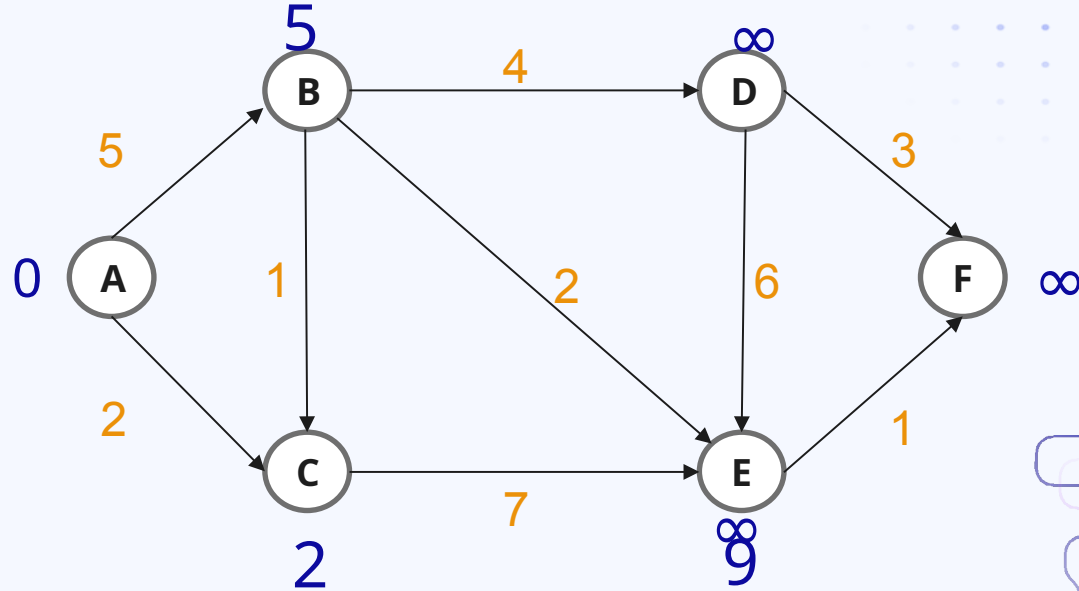
Dijkstra's Example

Nod e	Distance	Parent
A	0	-
B	5	A
C	2	A
D	∞	
E	∞	
F	∞	



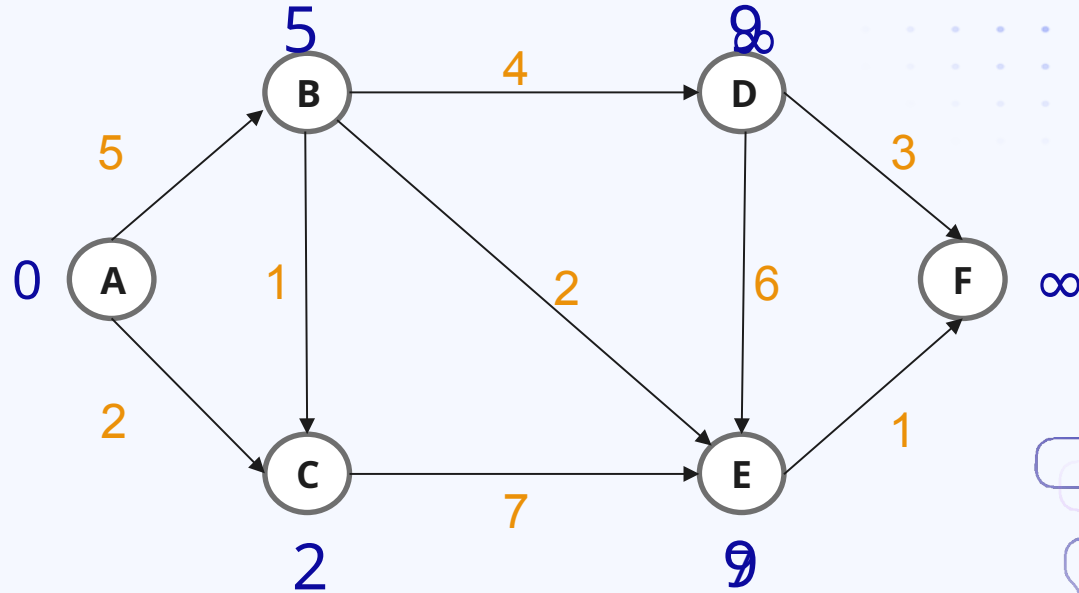
Dijkstra's Example

Nod e	Distance	Parent
A	0	-
B	5	A
C	2	A
D	∞	
E	9	C
F	∞	



Dijkstra's Example

Nod e	Distance	Parent
A	0	-
B	5	A
C	2	A
D	8	B
E	9	B
F	∞	

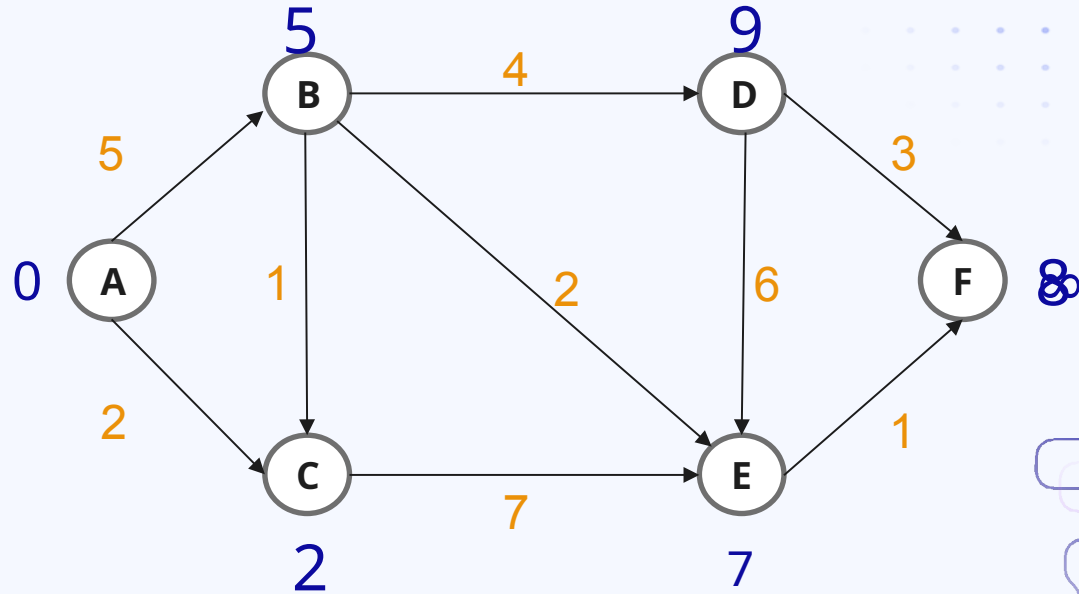


Dijkstra's Example

Nod e	Distance	Parent
A	0	-
B	5	A
C	2	A
D	9	B
E	7	B
F	∞	E

Visited list in order:

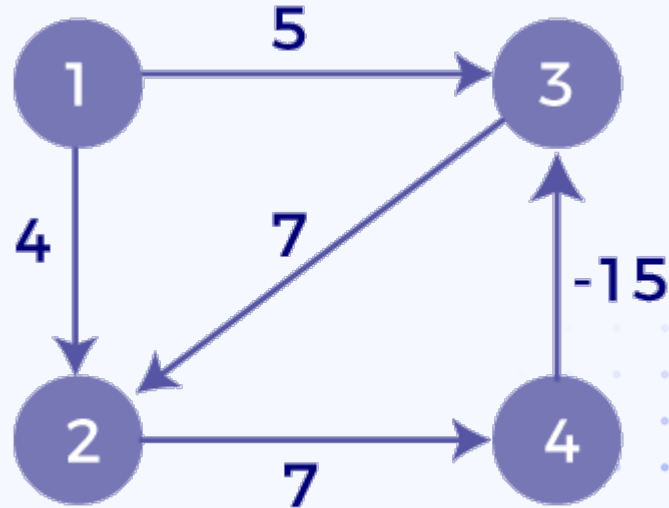
A C B E F D



Bellman-Ford

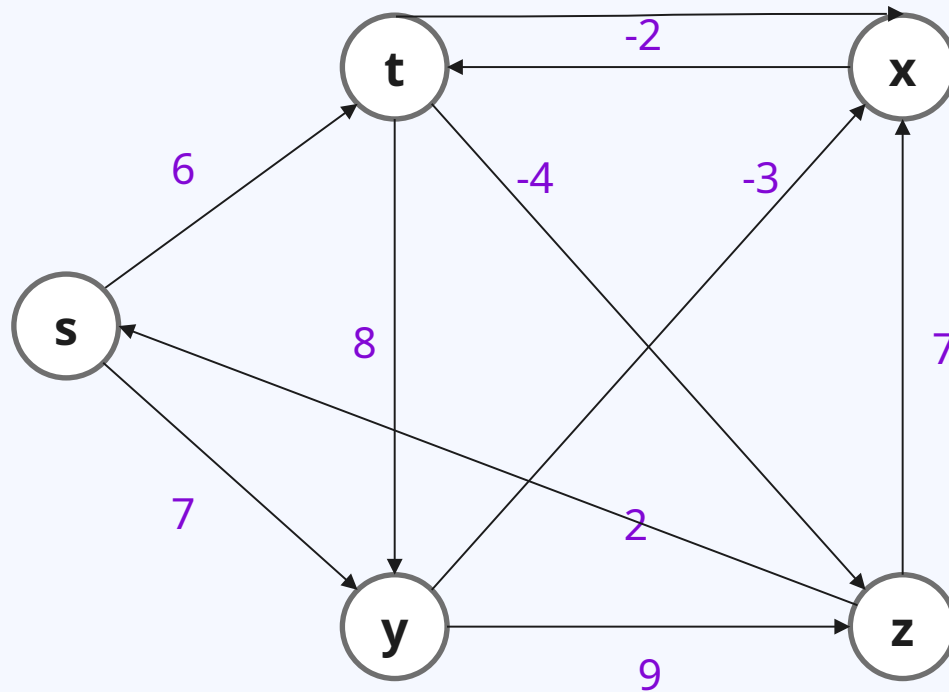
The Bellman-Ford algorithm works on graphs with **weighted edges** and can handle **negative weights**, provided **no negative weight cycles exist**.

It iteratively relaxes the edges, updating the shortest path estimates until convergence.



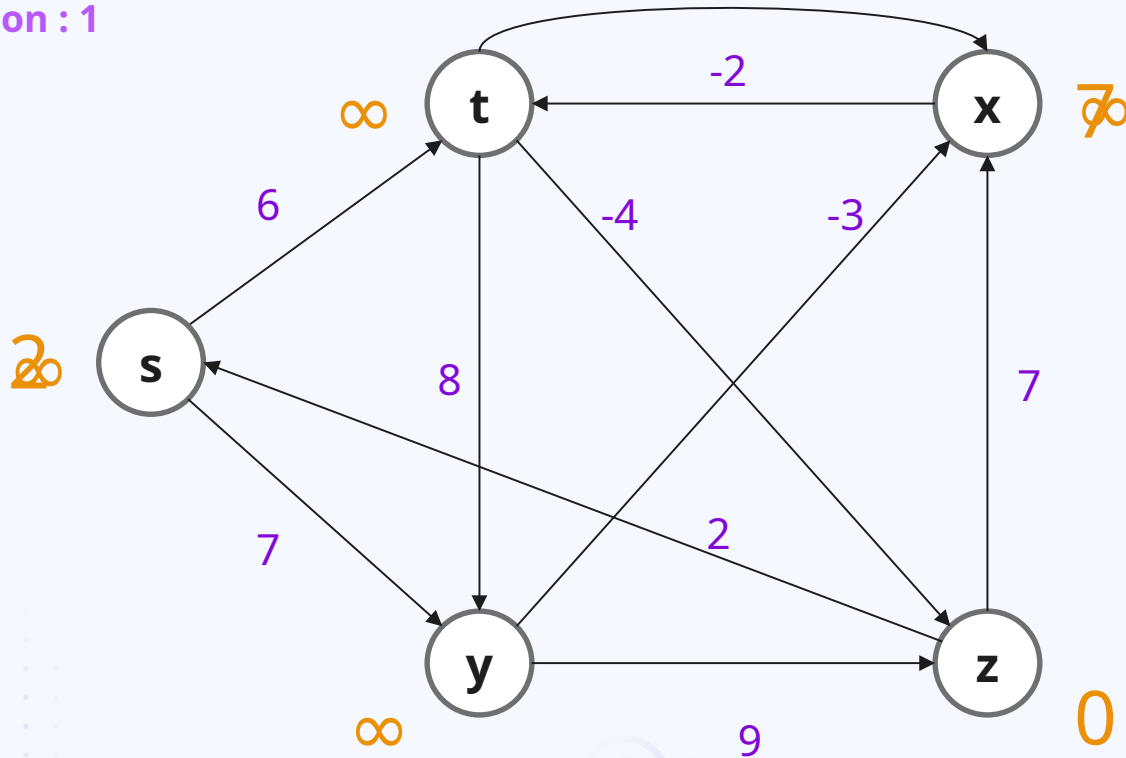
Bellman-Ford's Example

5



Bellman-Ford's Example⁵

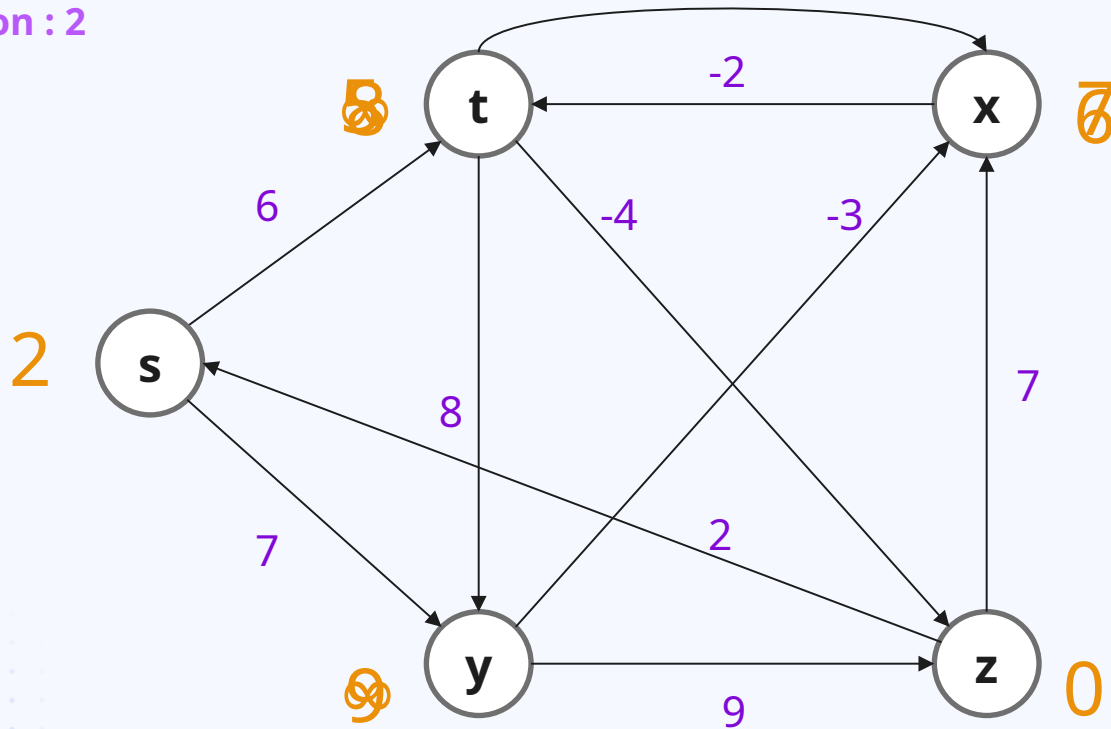
Iteration : 1



Bellman-Ford's Example

5

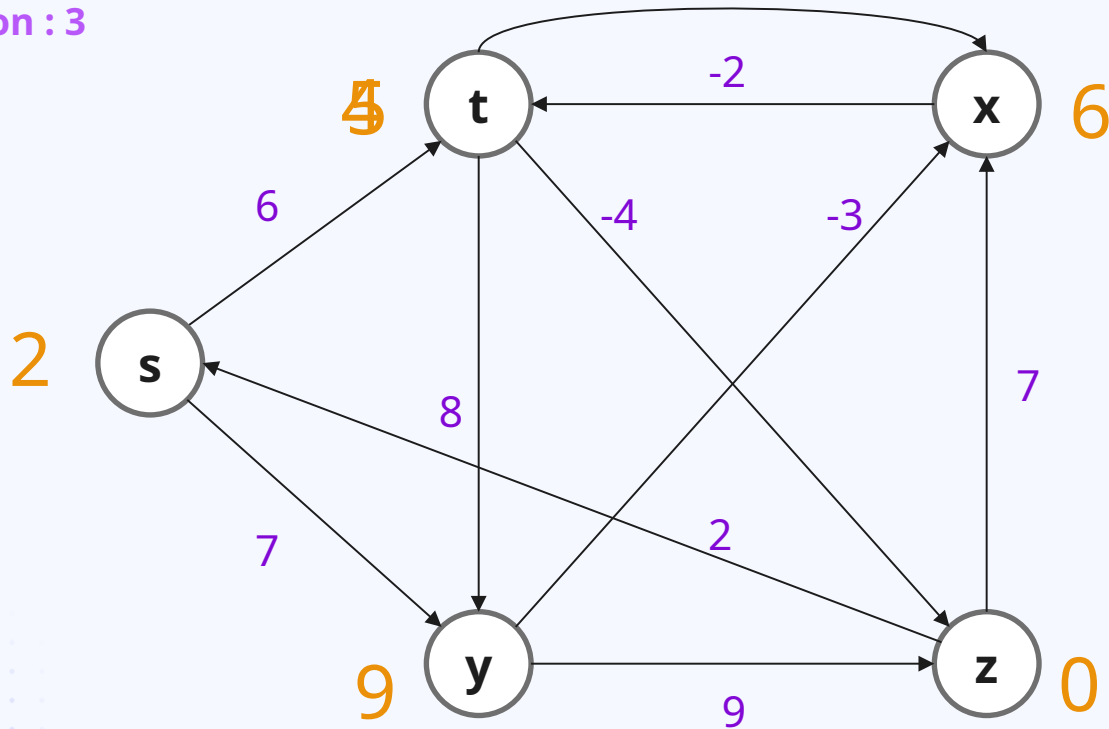
Iteration : 2



Bellman-Ford's Example

5

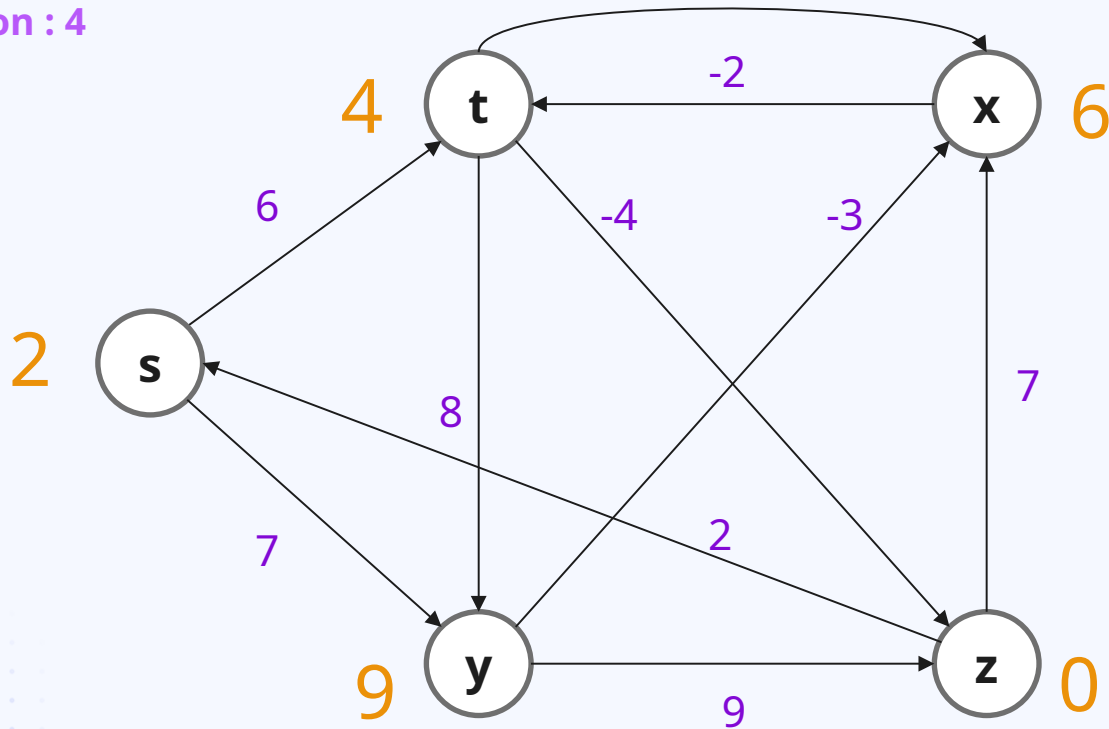
Iteration : 3



Bellman-Ford's Example

5

Iteration : 4



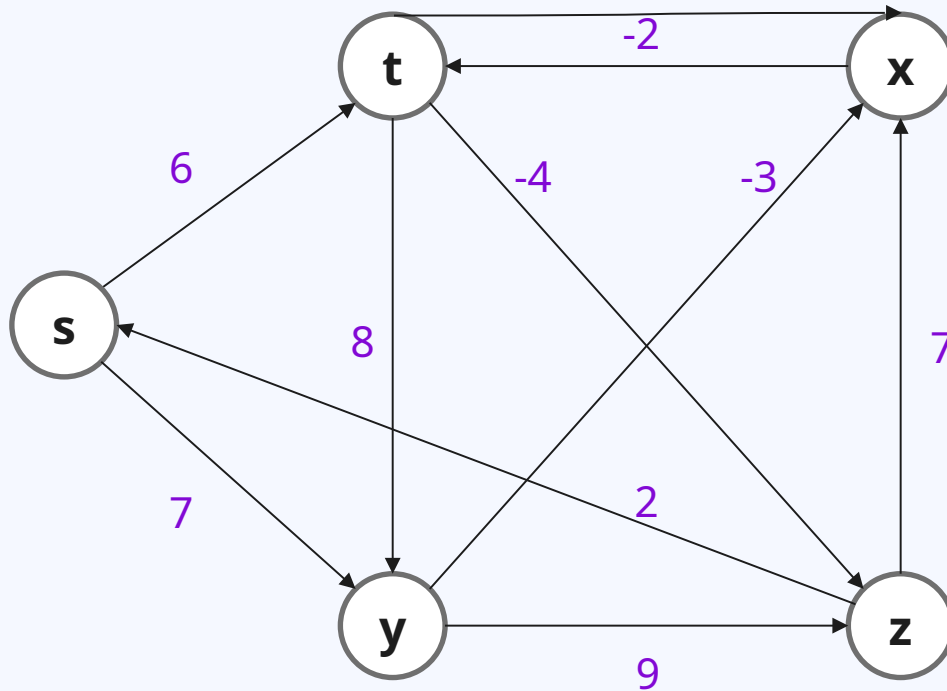
Johnson's Algorithm

(Dijkstra + Bellman Ford)

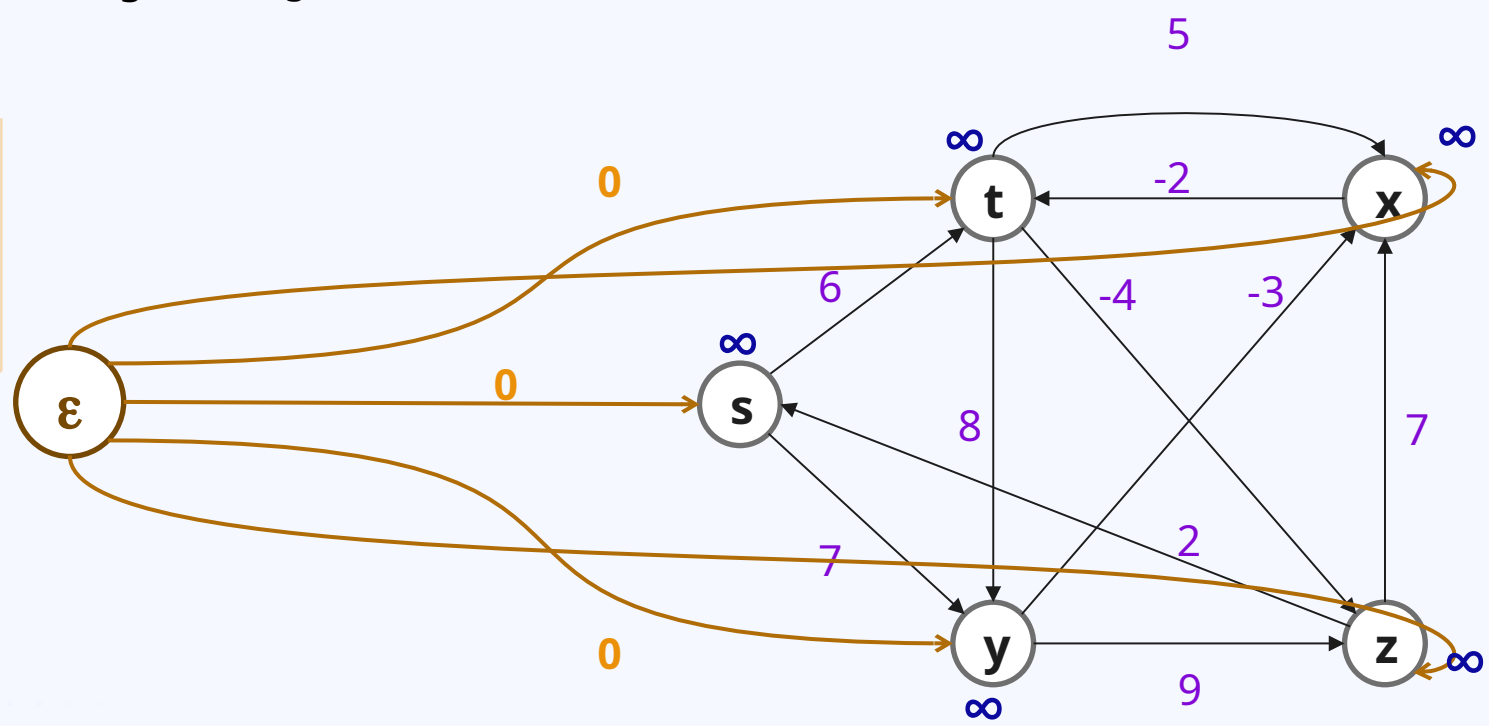
1. Adds an external node to connect all other nodes with zero-weighted edges.
2. Uses Bellman-Ford to establish distances from the external node.
3. Applies Dijkstra's algorithm for each node to derive the shortest paths.

Johnson's Algorithm's Example

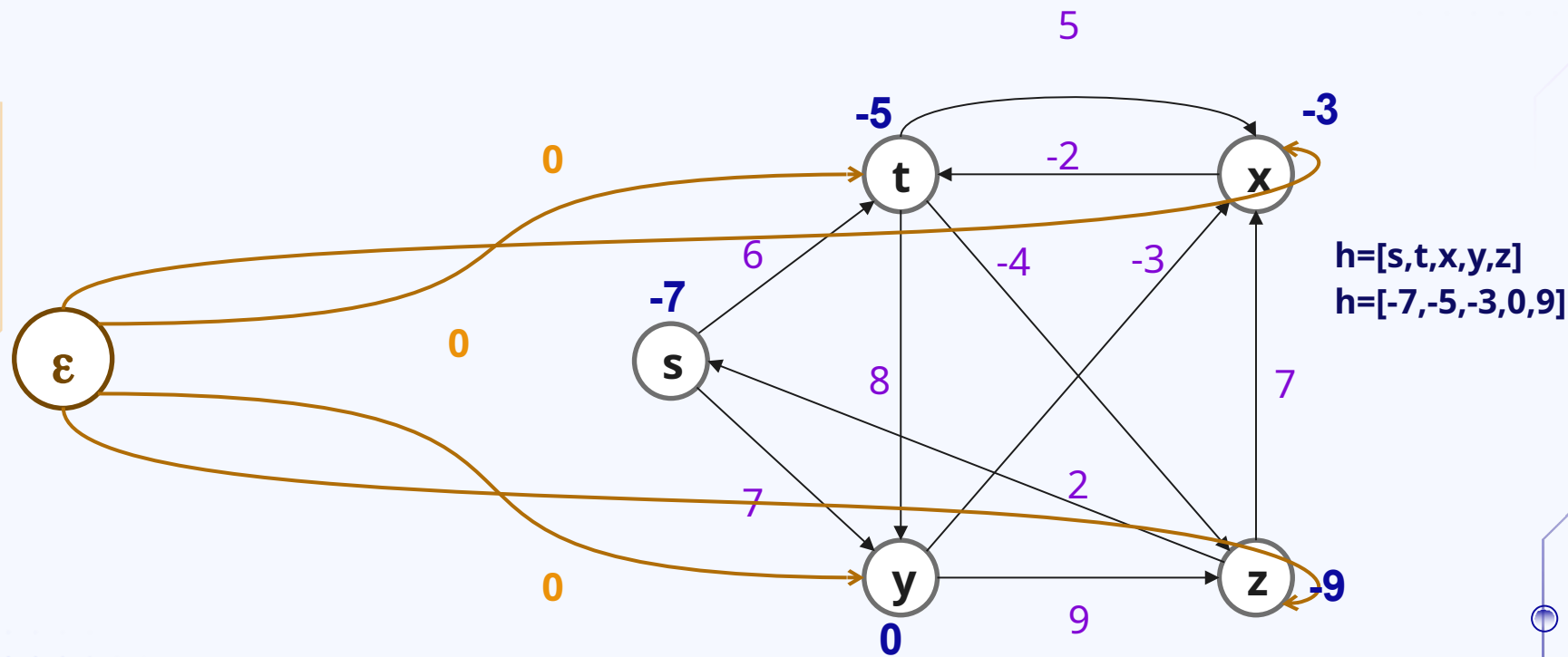
5



1. Adds an **external node** to connect all other nodes with **zero-weighted** edges.



2. Uses **Bellman-Ford** to establish distances from the external node.



3. Compute the **new weights** using the following formula:

$$D'(i,j) = D(i,j) + h[i] - h[j]$$

Where,

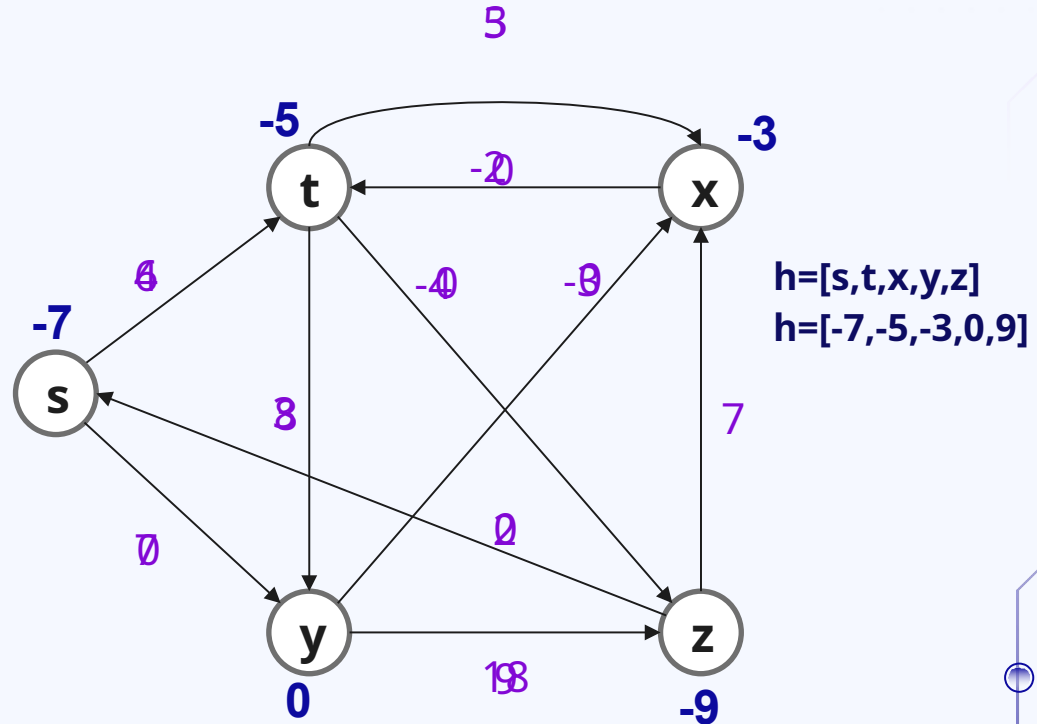
$D(i,j)$ is the old weight of the edge,

$h[i]$ is the source node value,

$h[j]$ is the destination node value.

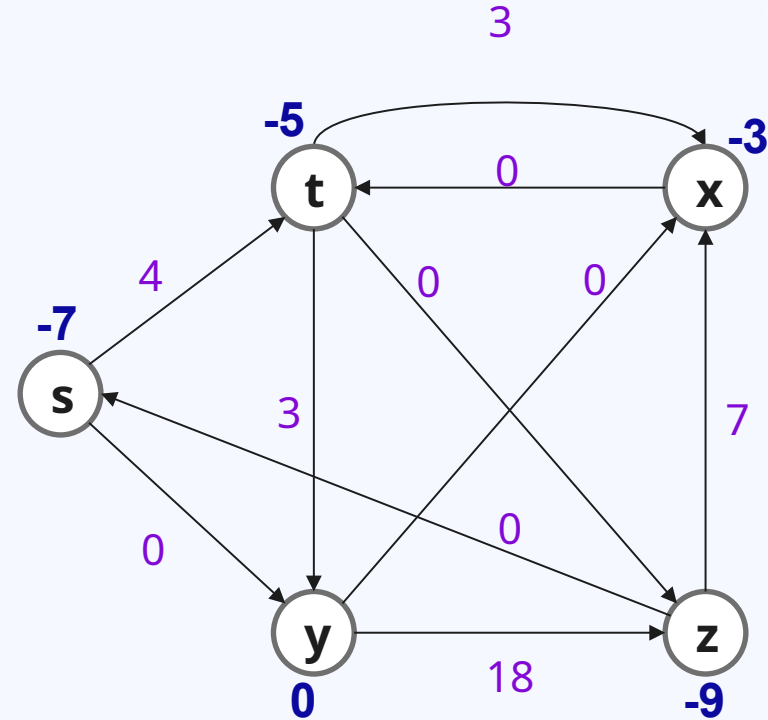
$$\begin{aligned} D'(i,j) &= D(i,j) + h[i] - h[j] \\ D'(s,t) &= D(s,t) + h[s] - h[t] \\ &= 6 + (-7) - (-5) \\ &= 4 \end{aligned}$$

$$\begin{aligned} D'(x,t) &= D(x,t) + h[x] - h[t] \\ &= -2 + (-3) - (-5) \\ &= 0 \end{aligned}$$



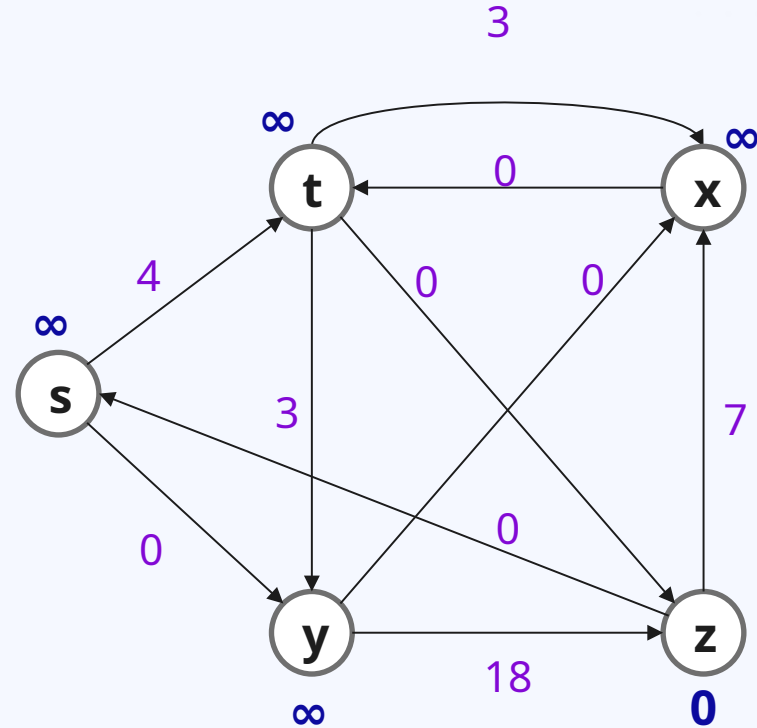
4. Now, all the edges are transformed to positive edges, run **Dijkstra** to find the shortest path, start from any node.

Node	Distance	Parent
z	0	-
s	∞ 0	z
t	∞ 4 0	x
x	∞ 7 0	y
y	∞ 0	s



5. After Dijkstra, shortest paths from Z are 0 for all nodes. Transform computed distances to original distances.

Node	Distance	Parent
z	0	-
s	∞ 0	z
t	∞ 4 0	x
x	∞ 7 0	y
y	∞ 0	s



5. Update the distance after each Dijkstra iteration according to the following equation:

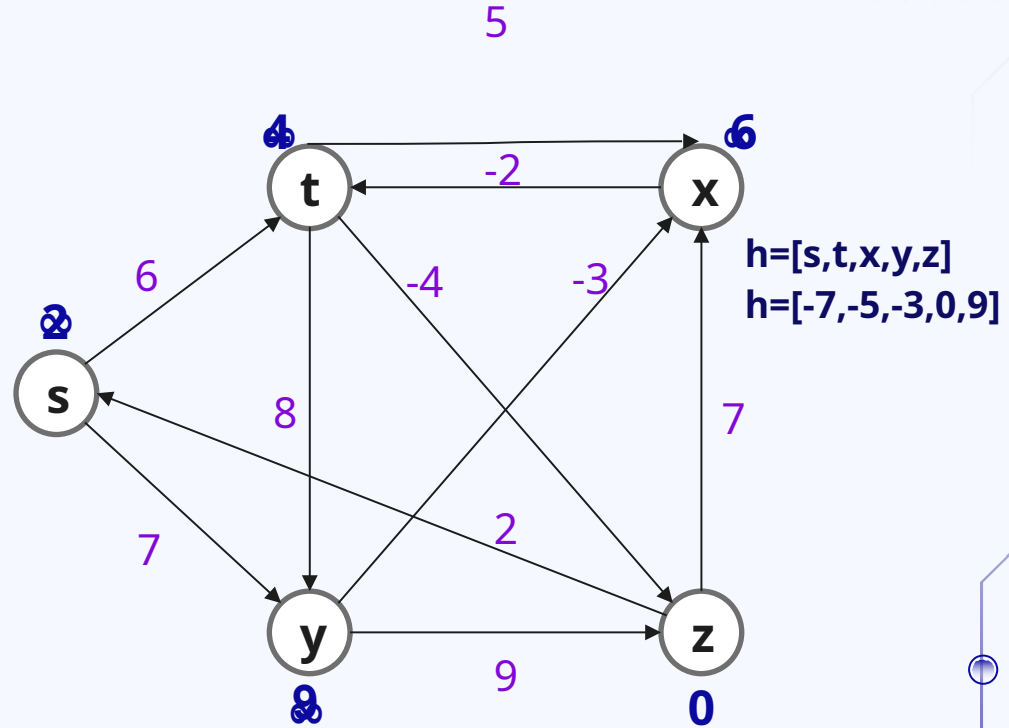
$$D'(i,j) = D(i,j) + h[i] - h[j]$$

$$D'(z,s) = 0 + (-7) - (-9) = 2$$

$$D'(z,t) = 0 + (-5) - (-9) = 4$$

$$D'(z,x) = 0 + (-3) - (-9) = 6$$

$$D'(z,y) = 0 + (0) - (-9) = 9$$



07 All Pairs Shortest Path

A decorative background featuring a light blue circuit board pattern with various colored lines (blue, orange, purple) and dots. The pattern is more dense on the right side of the slide.

Slow-All-Pairs-Shortest-Path

An algorithm that computes the **shortest path between all pairs of nodes**.

Calculate L^i matrix starting from $i=1$.

which indicates the shortest path containing 1 edge ,
till i = number of edges.

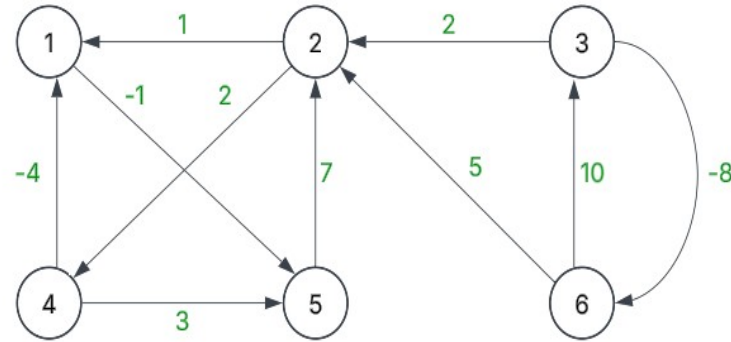
- For each matrix, $l_{ij} = \min(l_{ij}, l_{ik} + w_{kj})$ for all $k = 1$ to n .

Example

Adjacent matrix:

$$L^{(1)} = \begin{bmatrix} 0 & \infty & \infty & \infty & -1 & \infty \\ 1 & 0 & \infty & 2 & \infty & \infty \\ \infty & 2 & 0 & \infty & \infty & -8 \\ -4 & \infty & \infty & 0 & 3 & \infty \\ \infty & 7 & \infty & \infty & 0 & \infty \\ \infty & 5 & 10 & \infty & \infty & 0 \end{bmatrix}$$

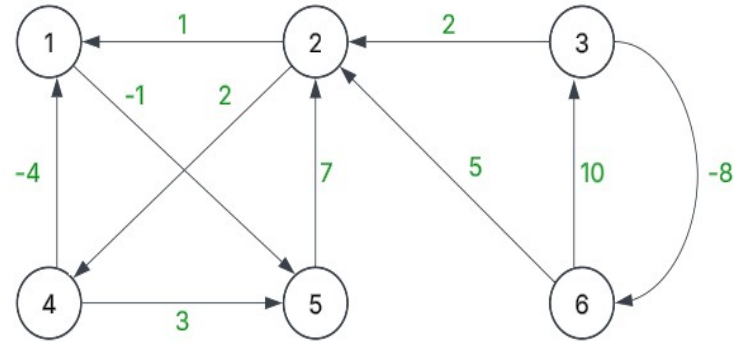
$$L^{(2)} = \begin{bmatrix} 0 & 6 & \infty & \infty & -1 & \infty \\ -2 & 0 & \infty & 2 & 0 & \infty \\ 3 & -3 & 0 & 4 & \infty & -8 \\ -4 & 10 & \infty & 0 & -5 & \infty \\ 8 & 7 & \infty & 9 & 0 & \infty \\ 6 & 5 & 10 & \infty & \infty & 0 \end{bmatrix}$$



Example

$$L^{(3)} = \begin{bmatrix} 0 & 6 & \infty & 8 & -1 & \infty \\ -2 & 0 & \infty & 2 & -3 & \infty \\ -2 & -3 & 0 & -1 & 2 & -8 \\ -4 & 2 & \infty & 0 & -5 & \infty \\ 5 & 7 & \infty & 9 & 0 & \infty \\ 3 & 5 & 10 & 7 & 5 & 0 \end{bmatrix}$$

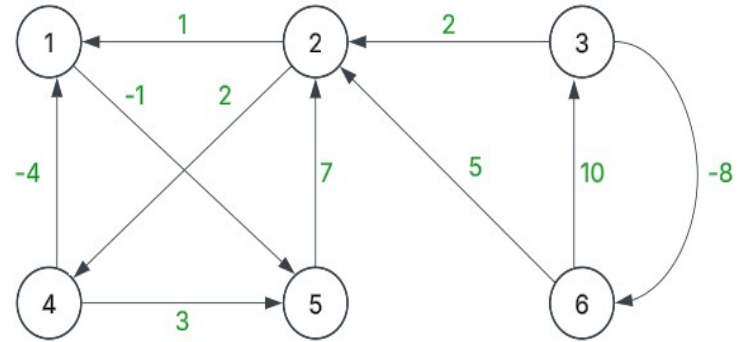
$$L^{(4)} = \begin{bmatrix} 0 & 6 & \infty & 8 & -1 & \infty \\ -2 & 0 & \infty & 2 & -3 & \infty \\ -5 & -3 & 0 & -1 & -3 & -8 \\ -4 & 2 & \infty & 0 & -5 & \infty \\ 5 & 7 & \infty & 9 & 0 & \infty \\ 3 & 5 & 10 & 7 & 2 & 0 \end{bmatrix}$$



Example

$$L^{(5)} = \begin{bmatrix} 0 & 6 & \infty & 8 & -1 & \infty \\ -2 & 0 & \infty & 2 & -3 & \infty \\ -5 & -3 & 0 & -1 & -6 & -8 \\ -4 & 2 & \infty & 0 & -5 & \infty \\ 5 & 7 & \infty & 9 & 0 & \infty \\ 3 & 5 & 10 & 7 & 2 & 0 \end{bmatrix}$$

$$L^{(5)}_{i,j} = \min(L^{(4)}_{i,k}, L^{(1)}_{k,j})$$



Fast-All-Pairs-Shortest-Path

The fast algorithm is the same as the slow one, **except** that the **matrices are computed for the powers of 2**

i.e. $L^{(1)}, L^{(2)}, L^{(4)}, L^{(8)} \dots$

For each matrix,
$$L^M_{i,j} = \min(L^{M-1}_{i,j}, L^{M-1}_{i,k} + L^{M-1}_{k,j})$$
 for all $k = 1$ to n .

Floyd-Warshall

All-Pairs Shortest Path Algorithm,

computes $D^{(i)}$ matrices,

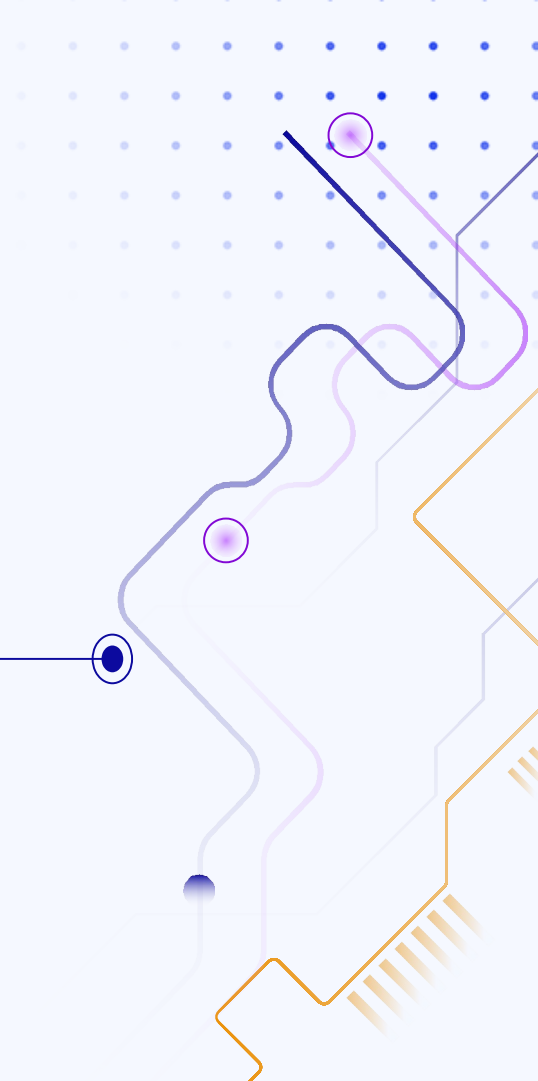
where $i = 1 \rightarrow$ Number of nodes.

$D^{(i)}$ computes the shortest path by setting $Node^{(i)}$ as the intermediate node.



08

Flow Networks

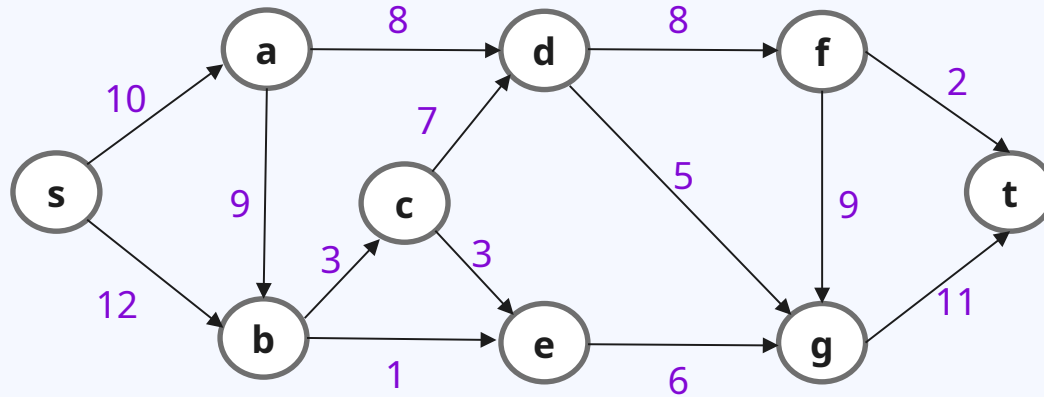


Flow Networks

Ford-Fulkerson algorithm to compute the **maximum flow** between s and t

Flow Network's Example

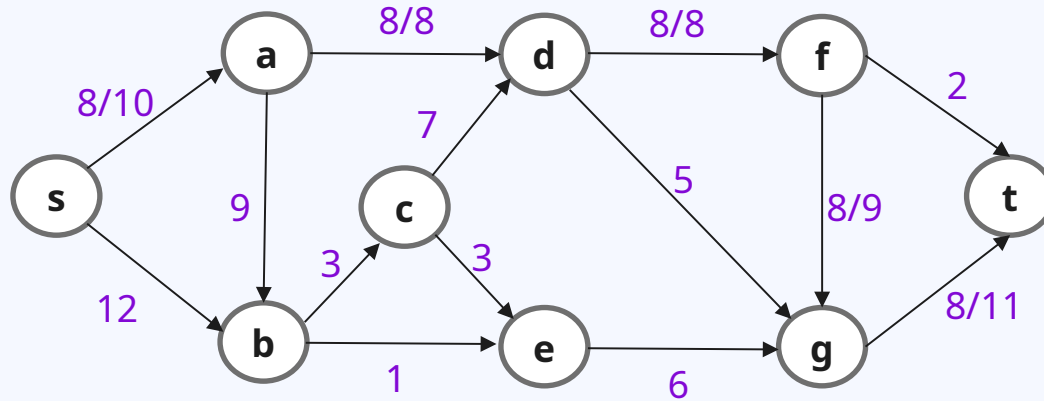
Iteration 1



Flow = 8

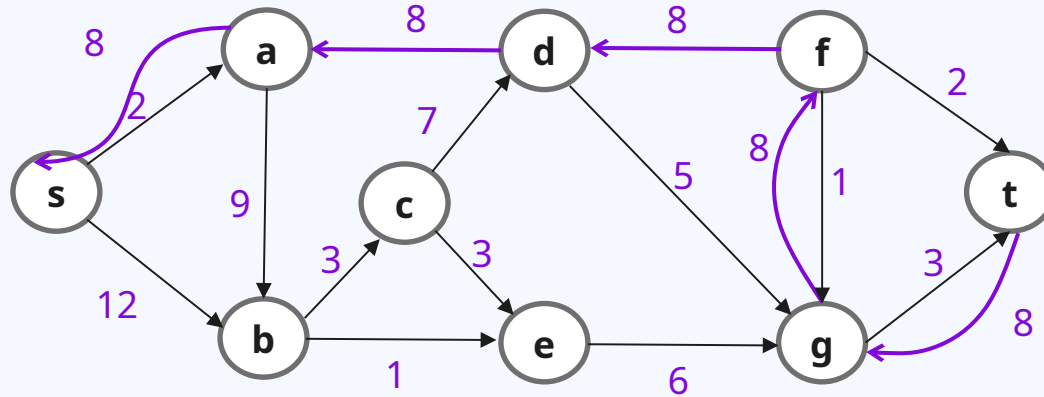
Flow Network's Example

Using path $s \rightarrow a \rightarrow d \rightarrow f \rightarrow g \rightarrow t$



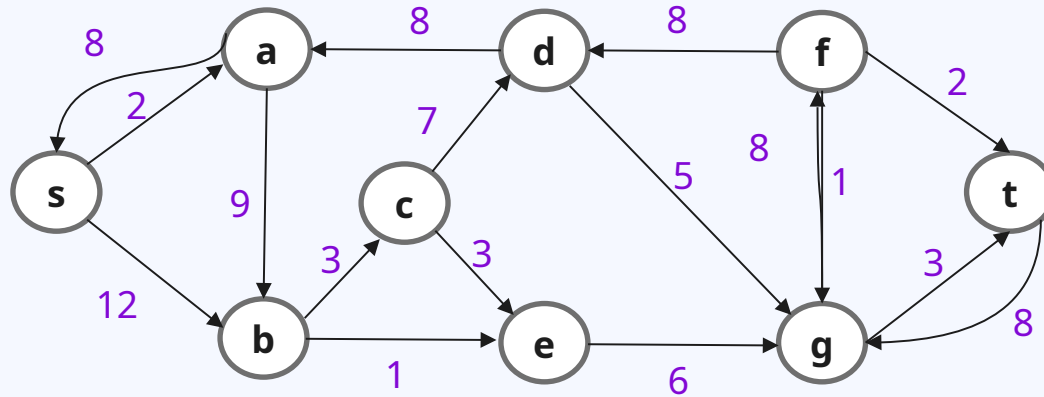
Flow Network's Example

Residual Network



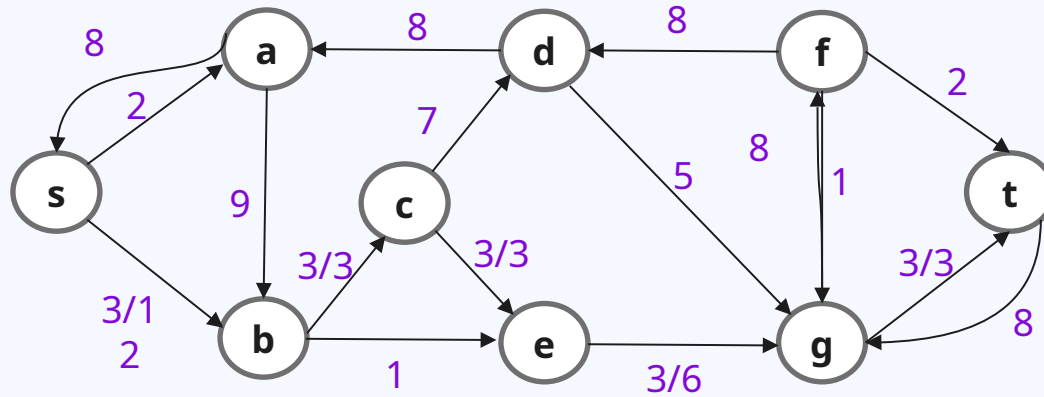
Flow Network's Example

Iteration 2



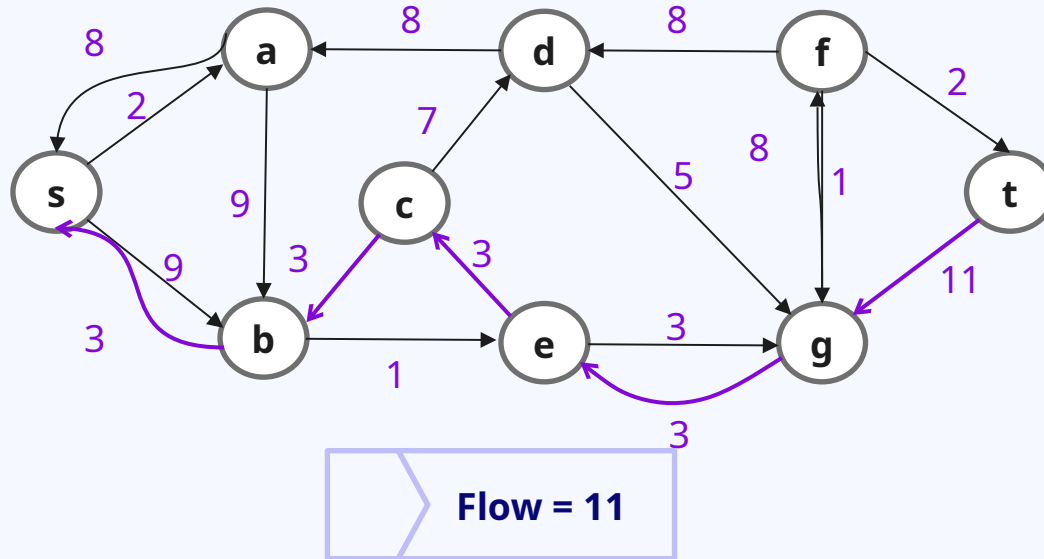
Flow Network's Example

Using path $s \rightarrow b \rightarrow c \rightarrow e \rightarrow g \rightarrow t$



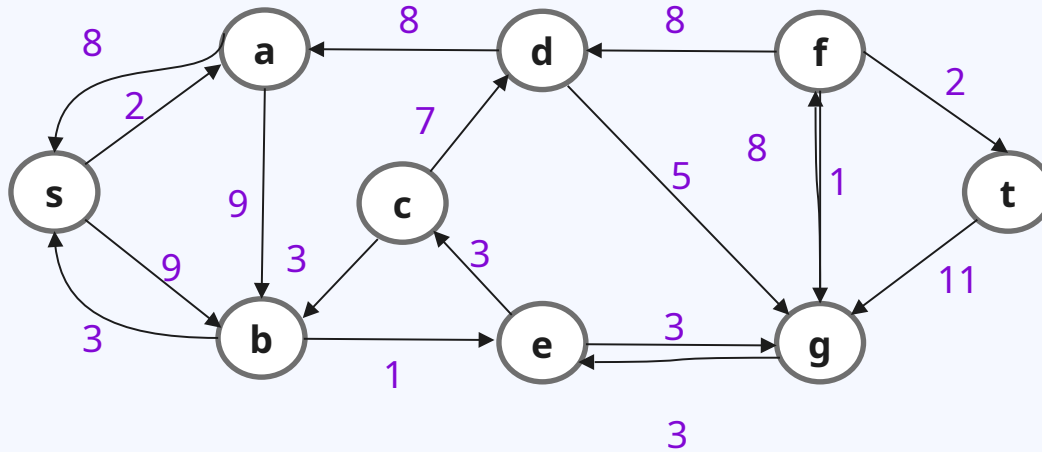
Flow Network's Example

Residual Network



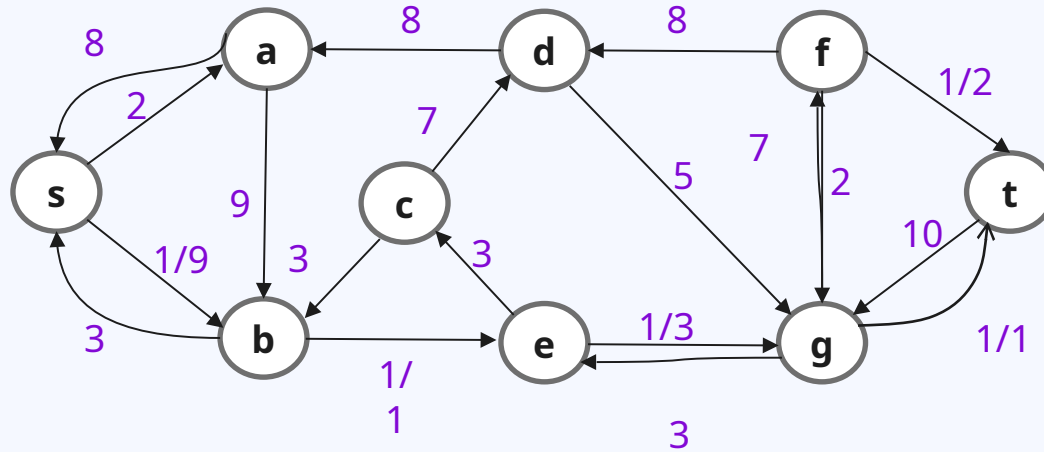
Flow Network's Example

Iteration 3



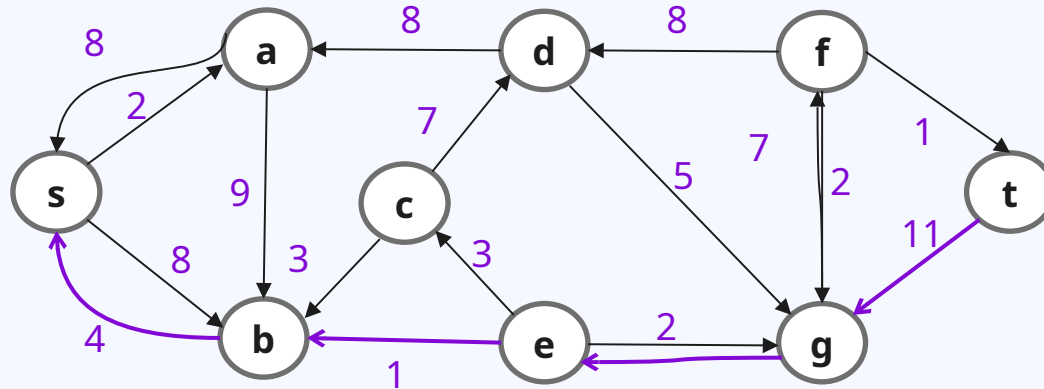
Flow Network's Example

Using path $s \rightarrow b \rightarrow e \rightarrow g \rightarrow f \rightarrow t$



Flow Network's Example

Residual Network



Flow = 12

Minimum Cut

A separator splits the graph to minimize the input from the source subgraph to the target one.

Minimum Cut' Example

